FinTrack AI

Proof of Concept

AI-POWERED PERSONAL FINANCE TRACKER FOR INDIA

This document is a proof of concept for the FinTrack AI application. Every capability described below is present in the project source or was verified live against the production backend during the build session. Items that are documented in the repository but not personally re-run in this session are attributed as such.

Field	Value
Prepared for	Dheeru Reddy (dheerureddy2004@gmail.com)
Date	7 July 2026
Platforms	iOS, Android, and Web from one Expo codebase
Frontend	React Native 0.86 / React 19 / Expo SDK 57 / Expo Router / TypeScript
Backend	Supabase (Postgres, Auth, Row-Level Security, Realtime, pgvector, Edge Functions)
Repository state	Fresh git repository, initial commit d96ac4c (81 files)
Android build	EAS preview APK, status FINISHED, build 7d03d355
App package	com.dheerureddy.fintrackai, version 1.0.0 (versionCode 2)

1. Executive summary

FinTrack AI is a cross-platform personal-finance application built for the Indian market. A single React Native + Expo codebase runs on iOS, Android, and the web. The backend is Supabase, providing Postgres with Row-Level Security, authentication, Realtime updates, and pgvector similarity search. All AI runs server-side in Supabase Edge Functions, so no model keys ever ship inside the app bundle.

The product's guiding principle is trustworthy intelligence: every quantitative answer a user sees is computed by SQL, never guessed by a language model. Forecasting, anomaly detection, and subscription detection are deterministic and explainable; the language model is used only to phrase the computed facts in plain English, and every AI path has a deterministic fallback.

This proof of concept documents the full working application and, in particular, an end-to-end Goals experience that was designed, built, and verified during this engagement, then packaged as an installable Android APK through EAS Build.

2. Problem and motivation

- Indians manage money across UPI, cards, and cash, and want low-friction logging plus insight in rupees and India-local time, without spreadsheet drudgery.
- Generic trackers force tedious manual entry, or lean on language models that confidently invent figures, which is unacceptable for personal finance.
- FinTrack AI targets both problems: four fast ways to capture a transaction, and an intelligence layer whose numbers are always computed deterministically and can be cited back to the underlying transactions.

3. Feature set

Authentication and security

- Email and password authentication via Supabase Auth with persisted sessions restored on launch (app/login.tsx, src/auth/AuthProvider.tsx).
- Row-Level Security enabled on every table so a user can only ever read or write their own rows (supabase/schema.sql).

Overview dashboard

- Rolling 30-day income, expense, and savings with change versus the prior 30 days, robust at the start of a month (rollingStats in src/data/analytics.ts).
- Balance-trend and spending-trend charts with a Weekly / Monthly / Yearly toggle (balanceTrend, spendingTrend).
- Budgets-and-goals summary, upcoming planned payments predicted from recurring charges, a spend-by-category donut, and a recent-records list, all updating live via Supabase Realtime.

Four ways to add a transaction

The Add screen (app/(tabs)/add.tsx) exposes four capture modes; each AI-assisted path has an on-device deterministic fallback, and low-confidence items are flagged for one-tap review:

- Manual entry form.

- Natural language (for example, "paid 799 to Airtel for wifi bill yesterday") parsed into amount, merchant, category, and date.
- Receipt or UPI-screenshot photo, read by a vision model (via expo-image-picker).
- Bank CSV import with batch categorization (src/lib/csv.ts).

Ledger and budgets

- Full ledger with search, filters, one-tap recategorize, and delete-with-confirm.
- Named budgets over any set of categories, on weekly, monthly, or yearly periods, each with "trending over" and "over budget" nudge toggles; budgets are editable.

Insights (deterministic, phrased by AI)

- Month-end budget-pacing forecast that blends the current pace with three months of history (forecastMonth).
- Anomaly detection: a charge at least three times the category median and above Rs 1,500 (detectAnomalies).
- Subscription detection: recurring same-amount charges, flagged "likely forgotten" after 40 days (detectSubscriptions).
- Findings are generated deterministically (generateInsights) and phrased by the insights Edge Function, with a template fallback if the model is unavailable.

Conversational RAG agent

The chat Edge Function answers questions about the user's finances. It plans the query, runs SQL aggregation functions for the exact figures (agg_summary, agg_by_category, agg_top_txns, agg_merchant), retrieves related transactions with pgvector, and has the model phrase a grounded answer with citations to the real transactions used. The figures come from SQL scoped to the signed-in user, never from the model.

Proactive nudges and notification centre

- A pure-SQL engine scans budgets and transactions for breaches, projected overshoots, statistical anomalies, and stale subscriptions.
- It runs daily via pg_cron and on every app open, made idempotent by dedupe keys, and delivers to an in-app notification centre with unread badges, live via Realtime.

Settings

- Profile, budgets, and accounts overview.
- Preferences: auto-archive completed goals, and celebrate completed goals (both persisted).
- Quick links to Goals and to the Alerts / notification centre with counts.
- Data and support: export transactions to CSV, help and feedback, rate the app, and privacy policy.
- System status (backend, data privacy) and sign out.

4. Goals experience (built and verified in this engagement)

The headline of this proof of concept is a complete Goals feature. It comprises a rich create/edit flow, contributions, an AI-grounded plan, a completion lifecycle with optional auto-archive, and manual archive, restore, and delete.

Create and edit

- A full-screen modal (app/goal-edit.tsx) with a quick-start preset grid: New vehicle, New home, Holiday trip, Education, Emergency fund, Health care, Party, Kids.
- Fields: name, target amount, saved already, desired date (with 3-month / 6-month / 1-year / 2-year quick chips and a clear control), goal colour (12 swatches), icon (22 options), and a note.
- A per-goal "auto-archive when complete" toggle, defaulting from the global preference.
- A synchronous re-entry guard (useRef) prevents a rapid double-tap from double-submitting.

Contributions and plan

- Quick contributions of Rs 500 / Rs 1,000 / Rs 5,000 and a custom-amount input.
- A feasibility pill (On track / Tight) and an AI-phrased monthly plan grounded in the user's real average monthly surplus (buildGoalPlan in src/ai/goals.ts).

Completion and lifecycle

- On reaching the target the goal shows a "Reached" state with a celebration banner.
- If auto-archive is on, the completed goal is moved automatically into a saved Archived section, removing it from the active list.
- Manual controls: Archive and save, Restore, and Delete (permanent, from the database, behind a confirmation).

Persistence design

The financial fields of a goal (name, target amount, current amount, deadline) are stored in the Supabase goals table, so they are synced and protected by Row-Level Security. The presentation and lifecycle fields (colour, icon, note, archived state, completion timestamp, and per-goal auto-archive) are stored in a local device overlay in AsyncStorage (src/data/goalMeta.tsx) and merged onto the database goal at read time via enrichGoal. This design was chosen because the live database could not take a schema migration in this environment; it needs no migration, requires no action from the user, and survives app reloads. The overlay is also read by the Overview (to hide archived goals and show each goal's colour and icon) and by Settings.

5. Architecture

The client renders with Expo Router and reads and writes server state through React Query over supabase-js, authenticated with the user's JWT. Supabase Postgres enforces Row-Level Security on every table, exposes a derived-balance view and SQL aggregation functions, provides pgvector similarity search, runs the nudge engine on a schedule, and streams changes over Realtime. Deno Edge Functions (embed, chat, insights, parse-txn) hold the only copies of the AI keys as function secrets, and every upstream model call is time-capped so a stalled connection cannot hang a request.

Layer	Responsibility
React Native + Expo Router	UI, navigation, four transaction-capture flows, goals, settings
React Query + supabase-js	Server-state cache, mutations, and live Realtime sync (user JWT)
Supabase Postgres	Tables with RLS, derived balance view, aggregation RPCs, pgvector, nudges
Edge Functions (Deno)	embed, chat, insights, parse-txn; AI keys live only here as secrets
AsyncStorage overlay	Local goal presentation and lifecycle metadata, plus app preferences

6. Verification and testing

Verified live in this session

- TypeScript check (npm run typecheck) passed with zero errors after every change.
- Against the production Supabase backend, signed in as the owner account on the Expo web build: created a goal from typed input (name, target, saved, note, date) and confirmed it persisted; made quick and custom contributions (a custom Rs 250 moved a goal from Rs 1,000 to exactly Rs 1,250); drove a goal to 100 percent and observed it auto-archive into the Archived section; restored and permanently deleted archived goals; confirmed edit pre-loads and saves.
- Exported transactions to CSV and confirmed a valid file of 161 lines (one header plus 160 rows).
- Toggled the Settings preferences and confirmed they persisted to storage.
- Observed zero browser-console errors throughout; all test goals were deleted and the account was restored to its original state afterwards.

Automated suites present in the repository

- npm run test:e2e is a backend end-to-end suite that runs against the deployed system and reports a pass/total assertion count. Its documented coverage includes authentication, Edge Function JWT gates, the transaction lifecycle, auto-embedding, the derived balance, two-account RLS isolation, AI ingestion contracts, hierarchical categories, budget CRUD, chat-versus-SQL ground truth, nudge idempotency, and goals CRUD.
- npm run eval is an LLM-as-Judge retrieval eval that grades the deployed agent against ground truth computed by the same deterministic SQL, with harness-fixed date ranges.

RAG eval results (evals/REPORT.md, run 6 July 2026, 13 cases)

Metric	Score
Numeric accuracy (deterministic check that expected figures appear)	13 / 13 (100%)
Correctness (Gemini judge, temperature 0; 9 judged)	9 / 9 (100%)
Faithfulness (no invented figures; 9 judged)	9 / 9 (100%)
Judge-unavailable (LLM quota; reported separately, not scored)	4 cases

Cases cover totals, category and date filtering, month-name parsing, merchant counting, comparisons, rankings, and savings math. Judge-unavailable runs are never counted as correct or incorrect.

7. Build and deployment

- Web preview: npm run web serves the app through Metro on port 8081; used for live verification.
- Android APK: built in the Expo cloud from the eas.json preview profile (internal-distribution APK). The build finished successfully; it reused the existing Android keystore and baked the public Supabase URL and publishable key into the binary.
- Result: package com.dheerureddy.fintrackai, version 1.0.0 (versionCode 2), installable via a QR code / download link from the EAS build page (build 7d03d355).
- The production profile is configured to build a Play-Store AAB with an auto-incrementing versionCode.

8. Security and privacy

- Row-Level Security is enabled on every table and scoped to the authenticated user id.
- AI keys exist only as server-side Edge Function secrets and are never bundled into the app; the client holds only the public Supabase URL and publishable anon key, which are safe to ship because RLS protects the data.
- The auto-embed function is gated by a shared secret; chat and insights require a valid JWT.
- The committed repository excludes secrets: the .env file is gitignored and the initial commit was verified to contain no .env or local-log files.
- Insights and nudges are descriptive analytics, not financial advice.

9. Technology stack

Area	Technology
Language	TypeScript (strict), plus SQL and Deno/TypeScript for Edge Functions
Frontend	React Native 0.86, React 19, Expo SDK 57, Expo Router, react-native-svg
State and data	TanStack React Query, supabase-js, AsyncStorage
Backend	Supabase: Postgres, Auth, Row-Level Security, Realtime, pgvector, Edge Functions
AI	Google Gemini / Gemma via Edge Functions, with on-device heuristic fallbacks
Build and delivery	EAS Build (preview APK, production AAB)

10. Honest limitations

- Goal colour, icon, note, and archived/completed state are stored on-device in AsyncStorage rather than in the database. They persist across reloads but not across devices; the financial figures do sync. A future database migration can move them server-side.
- Insights and nudges are descriptive, not financial advice.
- The EAS APK download link is time-limited; the build page can re-issue it.
- AI free-tier quotas can be exhausted; when that happens the app degrades gracefully to SQL-templated answers and on-device parsing.

11. Roadmap

- Push notifications for nudges (the in-app notification centre is already Realtime-live).
- Play Store submission via eas submit (the production profile is configured).
- Optional migration of goal presentation metadata into the database for cross-device sync.

A. Appendix: key references

Reference	Value
Initial commit	d96ac4c (81 files, no secrets)
EAS Android build	7d03d355-9c52-4cad-9f8c-d12659b51521 (FINISHED, preview APK)
Supabase project URL	https://yjwzhwvwciymjwujbjal.supabase.co
Goals source	app/goals.tsx, app/goal-edit.tsx, src/data/goalMeta.tsx, src/ai/goals.ts
Analytics source	src/data/analytics.ts (forecast, anomaly, subscription, trends)
Backend source	supabase/schema.sql, supabase/migrations/, supabase/functions/
Eval report	evals/REPORT.md (13 cases, run 6 July 2026)

Prepared as a proof of concept for FinTrack AI. All figures and behaviours above are drawn from the repository source or were verified live during the build session on 6 to 7 July 2026.